Node.js — Day 29

Final Revision + Quiz Notes

Long-term goal

These notes are a standalone reference. If you open them after 10 or 20 years, the explanations, mental models, examples, and rebuild sequence should let you understand the material without the original chat.

Purpose of Day 29

Day 29 is the final revision and assessment of the Node.js foundation. These are clean reference answers, not the learner's original quiz responses.

Node.js Foundation: Day 1 → Day 29 — COMPLETE

1. Command-Line Arguments and Validation

For node app.js Bhavana 26, the important indexes are:

```
argv[0] → Node executable path
argv[1] → app.js
argv[2] → "Bhavana"
argv[3] → "26"
```

process.argv is Node's array of command-line arguments. Arguments arrive as strings.

```
const age = Number(process.argv[3]);

if (Number.isNaN(age) || age < 0) {
    console.log("Invalid age");
    process.exit(1);
}
```

Number() converts numeric text into a number. **Number.isNaN(age)** detects NaN. **||** means OR, so either invalid condition causes rejection. For -5, the negative-age check is true, so the program prints the error and exits.

Remember

process.argv → command-line input. Number() → convert numeric text. Validation → protect the rest of the program from invalid input.

2. Promises, async/await, and Asynchronous Execution

```
function getUser() {
    return new Promise((resolve) => {
        setTimeout(() => resolve("Bhavana"), 2000);
    });
}

async function showUser() {
    const name = await getUser();
    console.log(name);
}

console.log("A");
showUser();
console.log("B");
```

Output:

```
A
B
Bhavana   // after approximately 2 seconds
```

async makes an async function always return a Promise. **await** pauses the current async function until the Promise provides its result; it does not freeze the entire Node.js process.

State	Meaning
pending	Operation is still in progress.
fulfilled	Operation completed successfully.
rejected	Operation failed.

Mental model

Promise = eventual result. async = function returns a Promise. await = pause this async function until the Promise provides its result.

3. HTTP Request and Response

```
POST /users
Content-Type: application/json

{
  "name": "Bhavana",
  "age": 26
}
```

Request part	Meaning
POST	HTTP method; commonly used to create a resource.
/users	Requested URL path.
Content-Type: application/json	The request body is being sent as JSON.
JSON object	Request body containing the data.

```
201
Content-Type: application/json

{
  "message": "User created"
}
```

201 is the response status. The Content-Type line is a response header. The JSON object is the response body.

Content-Type describes the format of data actually being sent. **Accept** expresses response formats the client can accept.

> **Easy rule**
>
> Request = what the client asks/sends. Response = what the server answers/sends.

4. Middleware and Error Handling

Middleware is a function/layer inside the server's request-processing pipeline.

```
Request
   ↓
Middleware 1
   ↓
Middleware 2
   ↓
Route Handler
   ↓
Response
```

next() means: the current middleware is done; continue to the next middleware or handler.

Middleware can perform logging, authentication, validation-related checks, or other request-level work. It can continue with next(), send a response and stop this request, or pass an error to error handling.

If middleware neither calls next() nor sends a response, the request has no path forward and can hang.

```
function authMiddleware(req, res, next) {
    if (!req.headers.authorization) {
        res.statusCode = 401;
        res.end("Unauthorized");
        return;
    }
    next();
}
```

throw new Error(...) explicitly signals an error. **console.log(...)** only prints a message; it does not throw an error.

> **Remember**
>
> next() = continue the request pipeline. throw = signal an error. res.end() = finish/send the response.

5. Environment Variables

Environment variables keep configuration outside the main source code. Common examples include ports, database connection details, API keys, and other environment-specific values.

```
PORT=3000
DB_PASSWORD=mySecretPassword
```

Node exposes environment variables through process.env:

```
process.env.PORT
process.env.DB_PASSWORD
```

.env	A file commonly used to store environment-variable values.
process.env	Node's environment object used to access environment variables.

Creating a .env file alone does not automatically mean every Node process loads it. Values must be loaded into the environment, for example using Node's --env-file option or another loading mechanism.

6. Modules: require() and module.exports

```
// user.js
function createUser(name, age) {
    return { name: name, age: age };
}

module.exports = createUser;
```

```
// app.js
const createUser = require("./user");

const user = createUser("Bhavana", 26);
```

module.exports exposes something from a module. **require()** loads the exported value/function.

./user identifies a local module/file path. An installed package is referenced by its package name.

> **Long-term memory**
>
> module.exports = make something available. require() = load/use what another module made available.

7. HTTP Status Codes — Quick Reference

Status	Meaning
200	OK / successful request
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

Memory: 200 = OK, 201 = created, 400 = bad request, 401 = authentication problem, 403 = access forbidden, 404 = not found, 500 = server-side failure.

8. Request Bodies in Raw Node.js HTTP

With Node's built-in http module, request bodies arrive as stream chunks. They are not automatically available as a parsed JavaScript object.

```
let body = "";

req.on("data", (chunk) => {
    body += chunk;
});

req.on("end", () => {
    const data = JSON.parse(body);
    console.log(data.name);
});
```

```
HTTP body
   ↓
chunks arrive
   ↓
collect chunks
   ↓
complete body text
   ↓
JSON.parse(body)
   ↓
JavaScript object
```

JSON.parse() converts JSON text into a JavaScript value/object.

> **Important**
>
> Do not assume raw Node's http module automatically gives you a ready-to-use req.body object.

9. Event Loop and Promise Microtasks

```
console.log("A");

setTimeout(() => {
    console.log("B");
}, 0);

Promise.resolve().then(() => {
    console.log("C");
});

console.log("D");
```

Output:

```
A
D
C
B
```

Synchronous code runs first. The Promise continuation is a microtask. The timer callback is processed later as a timer task; in this standard case the microtask runs before the timer callback.

> **Remember**
>
> setTimeout(..., 0) does not mean “run immediately.” It schedules the callback.

10. Final Architecture — Putting Node.js Together

```
Client
  ↓
POST /users + JSON body
  ↓
Node HTTP server
  ↓
Middleware
  ↓
Read / parse body
  ↓
Validation
  ↓
userService
  ↓
user.js / business logic
  ↓
Create user
  ↓
Serialize response JSON
  ↓
201 + headers + body
  ↓
Client
```

Layer	Responsibility
app/server	Receives requests and coordinates the flow.
Middleware	Request-level work such as logging/authentication/checks.
Validation	Checks whether incoming data is acceptable.
userService	Organizes user-related operations.
user.js	Contains user-related business rules and creates the user.
Response	Sends status, headers, and body back to the client.

Separating responsibilities keeps a backend easier to read, test, maintain, and extend.

11. Final Quiz — Clean Reference Answers

Q1 — process.argv: CLI arguments are strings. Number() converts numeric text. Number.isNaN() detects NaN. age < 0 rejects negative ages. || rejects when either condition is true.

Q2 — async/await: async makes the function return a Promise. await pauses the current async function for the Promise result without freezing the entire Node process.

Q3 — HTTP: POST is the method, /users is the URL path, the JSON is the request body, Content-Type describes its format, and 201 is appropriate after successful creation.

Q4 — Middleware/errors: next() continues the request pipeline. No next() and no response can leave a request hanging. throw signals an error; logging does not.

Q5 — Architecture: Request → server → middleware → body parsing → validation → service → business logic → response. Modules connect through exports and require.

12. Final Cheat Sheet

Concept	Remember
Node.js	JavaScript runtime outside the browser; uses V8.
process.argv	Command-line arguments; values arrive as strings.
__dirname	Directory containing the current file.
__filename	Complete path to the current file.
process.cwd()	Directory from which the Node process was started.
package.json	Declares project dependencies and metadata.
package-lock.json	Records the resolved dependency tree.
node_modules	Installed packages and their dependencies.
Callback	Function supplied to another function to be called later.
Promise	Represents an eventual asynchronous result.
async	Makes a function return a Promise.
await	Waits for a Promise result within the current async function.
Event Loop	Coordinates when ready asynchronous callbacks can be processed.
Middleware	A layer/function in the request-processing pipeline.
next()	Continue to the next middleware/handler.
module.exports	Expose functionality from a module.
require()	Load/use another module's exported value.
JSON.parse()	JSON text → JavaScript value/object.
JSON.stringify()	JavaScript value/object → JSON text.
201	Created — a resource was successfully created.

13. Rebuild the Foundation Years Later

If you return to these notes after 10 or 20 years, use this sequence instead of trying to memorize every sentence:

```
1. Node.js runs JavaScript outside the browser.
2. process.argv provides command-line input.
3. Node core APIs provide files, processes, HTTP, timers, etc.
4. npm manages project packages.
5. Async work uses callbacks, Promises, and async/await.
6. The Event Loop coordinates asynchronous callbacks.
7. Node's http module can create a server.
8. A request contains method, URL, headers, and possibly a body.
9. A JSON body can be parsed into a JavaScript value.
10. Middleware performs request-pipeline work.
11. Validation protects business logic from bad input.
12. Services organize domain operations.
13. Business logic contains application rules.
14. Modules communicate through exports and require.
15. The server sends status + headers + body in the response.
```

Architecture to remember

Client → HTTP request → server → middleware → parsing → validation → service → business logic → response → client

Final status

Node.js Foundation — COMPLETE

Day 29 was the final revision and assessment. The next phase is Express.js Day 1.

These notes intentionally prioritize whitespace, readability, examples, mental models, and future reconstruction over minimizing page count.